

Network Foundations

Yeah, Networking. This module explores computer networks, focusing on Local Area Networks and Wide Area Networks.

What is a Network?

A collection of interconnected devices that can communicate. Individual endpoint devices are usually called **nodes**, but they don't comprise the entire network by themselves.

Networks are important

Because they have enabled communication like nothing we had ever seen, making it possible to share resources, access to data and collaborate across great distances.

Local Area Networks connect devices over short distances, like within our homes.

It is typically owned and managed by a single person or organization and has high data transfer rates.

Wide Area Networks connect devices over greater distances.

Often owned and managed by several people and/or organizations. It has slower transfer rates due to long-distance data travel.

The Internet is the greatest example of a **WAN**, connecting millions of **LANs**.

When accessing the Internet from home, your **LAN** connects to your ISP's **WAN**, which grants your devices the ability to reach the Internet.

The **MoDem** (Modulator-Demodulator) plays a crucial role, acting as the bridge between home **LANs** and the **ISP's** infrastructure. It converts the digital signals that come from your router into a format that is transmissible over media like telephone lines and other cable systems, like fiber optics.

Network Concepts

Understanding these, the tools on which Networks are built, is important. The incredibly complex technology stack ubiquitous in our daily lives relies upon a set of well-defined and developed protocols and systems: the TCP/IP Stack.

OSI Model

Open Systems Interconnection Model

Curiously, I've always thought "OSI" was somebody's name.

It's a conceptual framework that standardizes functions of a telecommunication or computing system into seven abstract layers. It helps vendors and developers create interoperable devices and software.

Layer 1 - Physical Layer

It's the lowest layer and is responsible for the actual transmission of data. Raw bitstreams (streams of 1's and 0's, energy/no-energy, light/no-light, etc.) over a given medium (copper wire, thin air, fiber optic cables, etc.).

This layer deals with the hardware components of a network, like Ethernet Cables, and it's physical connections.

Layer 2 - Data Link Layer

Provides node-to-node data transfer. It's responsible for ensuring data frames are transmitted properly, with synchronization, error detection, and correction. Switches and bridges operate at this layer, using **MAC** Addresses to identify devices.

A **MAC** Address is a unique identifier assigned to a Network Interface Controller to be used as an address in a network, for communicating within a network segment. It is used in Ethernet, Wi-Fi, Bluetooth and other protocols.

Recognizable as a group of 6 pairs of hexadecimal digits, separated by hyphens, colons or nothing at all.

Example: 14:DB:1C:09:3F:A8

Layer 3 - Network Layer

Handles packet forwarding, including the routing of these through different routers in order to reach the desired destination network. It figures out logical addresses and determines the path towards its destination, ensuring it gets there across multiple networks. They utilize **IP** addresses to identify devices and determine its path.

An **IP** address is also a unique identifier that allows information to be sent between devices in different networks.

In its version 4 shape, it is recognizable as a group of 4 8-bit (0-255) numbers, separated by periods.

Example: 192.168.0.1

There are different types of **IPs**, Public and Private **IP** addresses.

Public IP addresses are associated to a network and thus differentiate it from others globally. It is provided to you by your **ISP**.

Private IP addresses are device-level addresses that differentiate them in a **LAN** setting. These are given out by your router to all devices on the network.

Layer 4 - Transport Layer

Provides end-to-end communication services to be used by applications. Responsible for the reliable (or unreliable) delivery of data, segmentation and reassembly of messages, flow control, and error checking.

TCP (Transmission Control Protocol) and **UDP** (User Datagram Protocol) work at this layer.

TCP is reliable, with connection-oriented transmissions with error recovery.

UDP is faster, lightweight, and connectionless. Though it does not guarantee delivery.

Layer 5 - Session Layer

Manages sessions between applications. Sessions are ongoing communications between devices. So this layer allows it to establish, maintain and terminate such connections.

It is essential for checkpointing and recovery of sessions so data transfers can resume after interruptions.

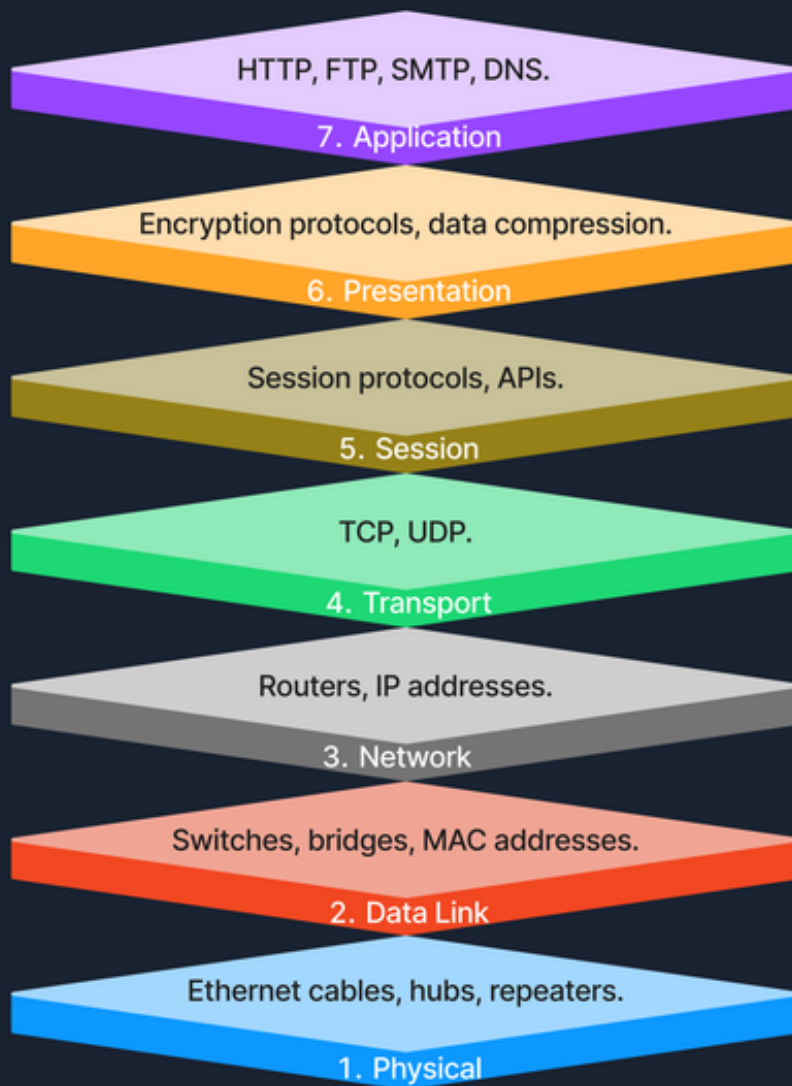
Layer 6 - Presentation Layer

It is the translator between the application layer and the network format. This includes data encryption and decryption, compression and conversion between data formats. Encryption and data compression

techniques operate here to secure and optimize transmissions.

Layer 7 - Application Layer

It is the topmost layer of the model. It includes common protocols such as **HTTP** (Hypertext Transfer Protocol), **FTP** (File Transfer Protocol), **SMTP** (Simple Mail Transfer Protocol) and **DNS** (Domain Name System), to name a few. It is the actual interface between the network and the applications that run on devices.



Example of Sending a File Across Network Layers

Several steps occur across different layers of the network model for that to happen.

At the **Application Layer**, the file transfer request is initiated. The **Presentation Layer** encrypts the file. The **Session Layer** establishes a communication session with the receiving end. The **Transport**

Layer breaks down the file into segments that ensure error-free transmission. The **Network Layer** determines the best route for transferring each segment of data. The **Data Link Layer** encapsulates the data into frames. And finally, the **Physical Layer** handles the actual transmission of bits over medium.

TCP/IP Model

It is a condensed version of the OSI model, with some layers shaven off. Tailored for more practical implementations on the internet.

Layer 1 - Link Layer

It is a join between Layers 1 and 2 of the OSI model. So it handles the physical delivery of data as well as it's framing.

Layer 2 - Internet Layer

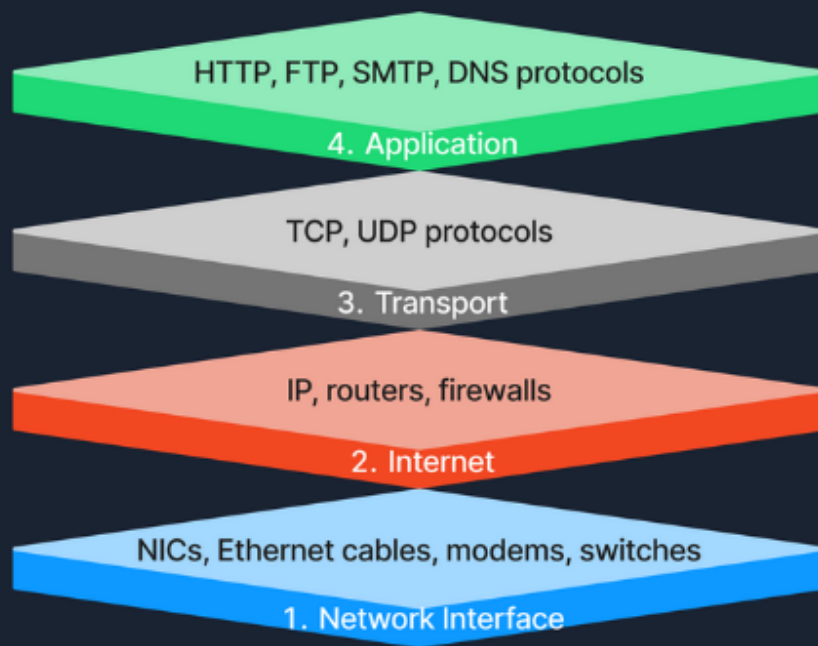
Translates directly to the **Network Layer** on the OSI model, so it handles logical addressing of devices via **IP**, and routes packets accross networks.

Layer 3 - Transport Layer

With the exact same name and function of the **Transport Layer** (Layer 4) of the OSI model, this layer is responsible for establishing reliable or fast communication with the **TCP** and **UDP** protocols. It ensures packets are delivered in an error-free and sequential manner.

Layer 4 - Application Layer

Encompassing all functions from the top 3 layers of the OSI model, those being the **Application**, **Pre-sentation** and **Session** layers, the **TCP/IP's Application Layer** then handles session establishment, recovery and termination, as well as cryptography and compression. But most importantly, it is where protocols such as **HTTP**, **FTP**, and **SMTP** work.



The TCP/IP model is a far more practical approach than the OSI model, but both play a role. While one is more practically implementable, the other offers a great theoretical framework, providing clearer insights on how networks function.

Protocols

Are standardized rules that determine formatting and processing of data to facilitate communication between devices. Protocols operate at different layers of the models, each tailored for a specific application and need, but always with that same objective.

The most common are:

- **HTTP** (Hypertext Transfer Protocol) Primarily used for transferring web pages. It operates at the Application Layer, allowing browsers and servers to communicate in the delivery of web content.
- **FTP** (File Transfer Protocol) Facilitates the transfer of files between systems, also functioning at the Application Layer. It provides a way for users to upload or download files to and from servers.
- **SMTP** (Simple Mail Transfer Protocol) Handles the transmission of email. Operating at the Application Layer, it is responsible for sending messages from one server to another, ensuring they reach their intended recipients.
- **TCP** (Transmission Control Protocol) Ensures reliable data transmission through error checking and

recovery, operating at the Transport Layer. It establishes a connection between sender and receiver to guarantee the delivery of data in the correct order.

- **UDP** (User Datagram Protocol) Allows for fast, connectionless communication, which operates without error recovery. This makes it ideal for applications that require speed over reliability, such as streaming services. UDP operates at the Transport Layer.
- **IP** (Internet Protocol) Crucial for routing packets across network boundaries, functioning at the Internet Layer. It handles the addressing and routing of packets to ensure they travel from the source to the destination across diverse networks.

Transmission

It is also important to know some things here.

There are two main types of transmission: **Analog** and **Digital**.

Analog is the more traditional way, using continuous signals to represent information, seen in radio broadcasts.

Digital employs discrete signals, transmitting data in bits. Typical in modern communication infrastructure.

Transmission Modes

Define how data is sent between two devices.

Simplex mode allows one-way communications only, from a keyboard to a computer, for example.

Half-duplex mode permits two-way communication, but in an alternating manner. Walkie-Talkies are an example.

Full-duplex is used in telephone calls for example and supports two-way simultaneous communication.

Transmission Media

These are the physical ways that data can be transmitted.

Wired media includes **Twisted Pair** cables, commonly used in **LANs**; **Coaxial** cables, used for TV and the early Ethernet; and **Fiber Optics**, which encodes data as light pulses, essential for high-speed connections.

Wireless media uses **Radio Waves** for Wi-Fi and cellular networks, **microwaves** for satellite comms, and infrared for remote controls, which are short-ranged.

Components of a Network

We have talked about the nodes on a network, but they can be many things and can be better defined further.

End Devices

Also known as **Hosts**. Any device that ends up sending or receiving data within a network.

Examples: Computers and smart devices (like phones and TVs).

Intermediary Devices

Facilitate data flow between **end devices**, within a network or between networks. These play a crucial

role in ensuring efficient and secure data transmission.

Examples: Routers, Switches, Modems, Firewalls and Access Points.

Intermediary Devices are responsible for the routing and addressing of packets, thus controlling the traffic within a network and between networks. They work at the Layer 2 and 3 of the OSI model.

Network Interface Card

A hardware component on a computer, or other, device that enables connection to a network.

Provides the physical interface between a device and the transmission media, sending and receiving data.

Each **NIC** has a unique **MAC** Address, essential for letting devices identify each other.

NICs are designed for each medium, such as Wi-Fi adapters for picking up Wi-Fi radio waves, and Ethernet cards for wired connection.

Routers

An Intermediary Device that routes packets between networks, allowing traffic to reach its intended destination. It operates at Layer 3 (Network Layer) of the OSI model.

They use protocols such as **Open Shortest Path First (OSPF)** or **Border Gateway Protocol (BGP)** to find the most efficient paths between nodes.

Routers can also increase security with functions like firewalls and control access lists. And they also help traffic flow by preventing congestions.

Switches

A switch's primary job is to connect devices within, typically, a **LAN**. They operate at the Data Link Layer (Layer 2) of the OSI model and thus use **MAC** addresses to forward data.

Hubs

A now antiquated networking device. It connects devices within a **LAN** in a simpler, non-intelligent, manner than switches. They work at Layer 1 of the OSI model, simply receiving signals and retransmitting them to all connected devices.

This behavior often results in collisions and inefficiencies.

Servers

Powerful computers used to serve and process applications to many other end devices, known as clients in this context.

Network Communication

Ok this might get hairy.

Crucial components of Network Communications are **MAC** and **IP** addresses, as well as **Ports**. Together, they ensure correct data delivery locally and globally.

MAC Addresses

A unique identifier assigned to the **NIC** of a device (so a device may have multiple **MACs** with multiple **NICs**), allowing it to be recognized on a **LAN**.

MAC addresses are 48 bits long, represented in Hex, appearing as six pairs segmented by colons or hyphens.

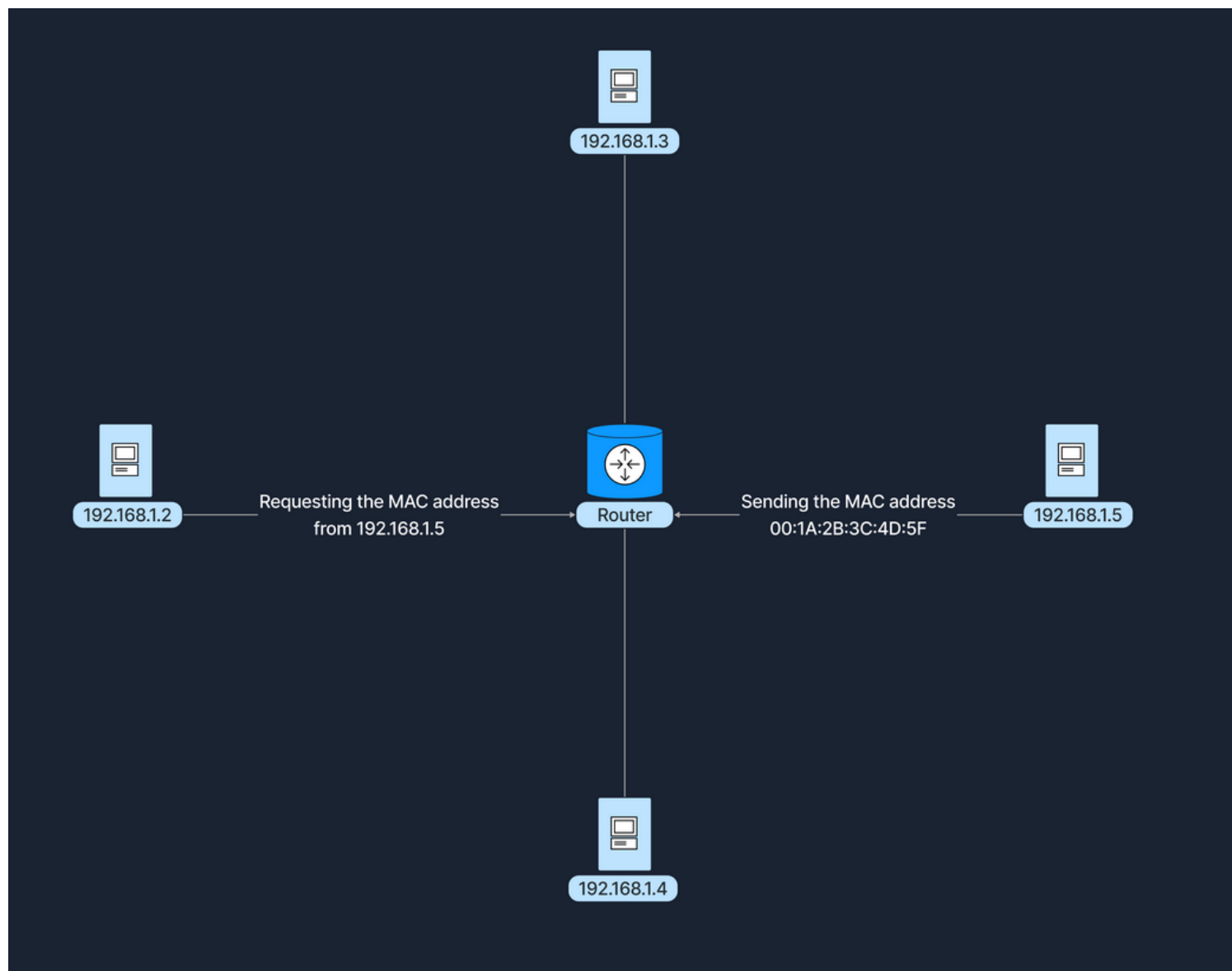
00:1A:2B:3C:4D:5E

The first 24 bits represent the Organizationally Unique Identifier (OUI) assigned **to** the manufacturer, while the other 24 are specific to the device. This ensures **MACs** are globally unique.

They are used to deliver data frames to the correct physical device.

A communication between two devices within a network works roughly as follows:

Computer A (with an **IP** address of 192.168.1.2) and Computer B (192.168.1.5), are connected to the same network switch. Computer A has the **MAC** address **00:1A:2B:3C:4D:5E**, while Computer B's **MAC** address is **00:1A:2B:3C:4D:5F**. When Computer A wants to send data to Computer B, it first uses the **Address Resolution Protocol (ARP)** to discover Computer B's MAC address associated with its **IP** address. After obtaining this information, Computer A sends a data frame with the destination **MAC** address set to 00:1A:2B:3C:4D:5F. The switch receives this frame and forwards it to the specific port where Computer B is connected, ensuring that the data reaches the correct device.



IP Addresses

A numerical label assigned to each device connected to a network.

I'm tired of typing shit out, here's the link to this module:

[Network Communication | Hack The Box Academy](#)

DHCP

Real quick on DHCP

It's an automatic local IP assignment protocol.

The acronym for the process it uses is DORA

Discover

When a new client connects to the network, it sends out a Discover packet, searching for the DHCP server so as to get leased an IP address.

Offer

The DHCP server receives the Discover packet and responds with an Offer, containing the leased IP.

Request

Is the packet sent by the client when accepting the Offer, confirming to the DHCP server that the IP lease has been accepted.

Acknowledge

Is the packet sent back to the client in response to the Request, stating that the server acknowledges the lease acceptance, and finalizing the exchange.

The Request -> Acknowledge cycle will repeat every time the IP lease nears its expiry date/time.



NAT

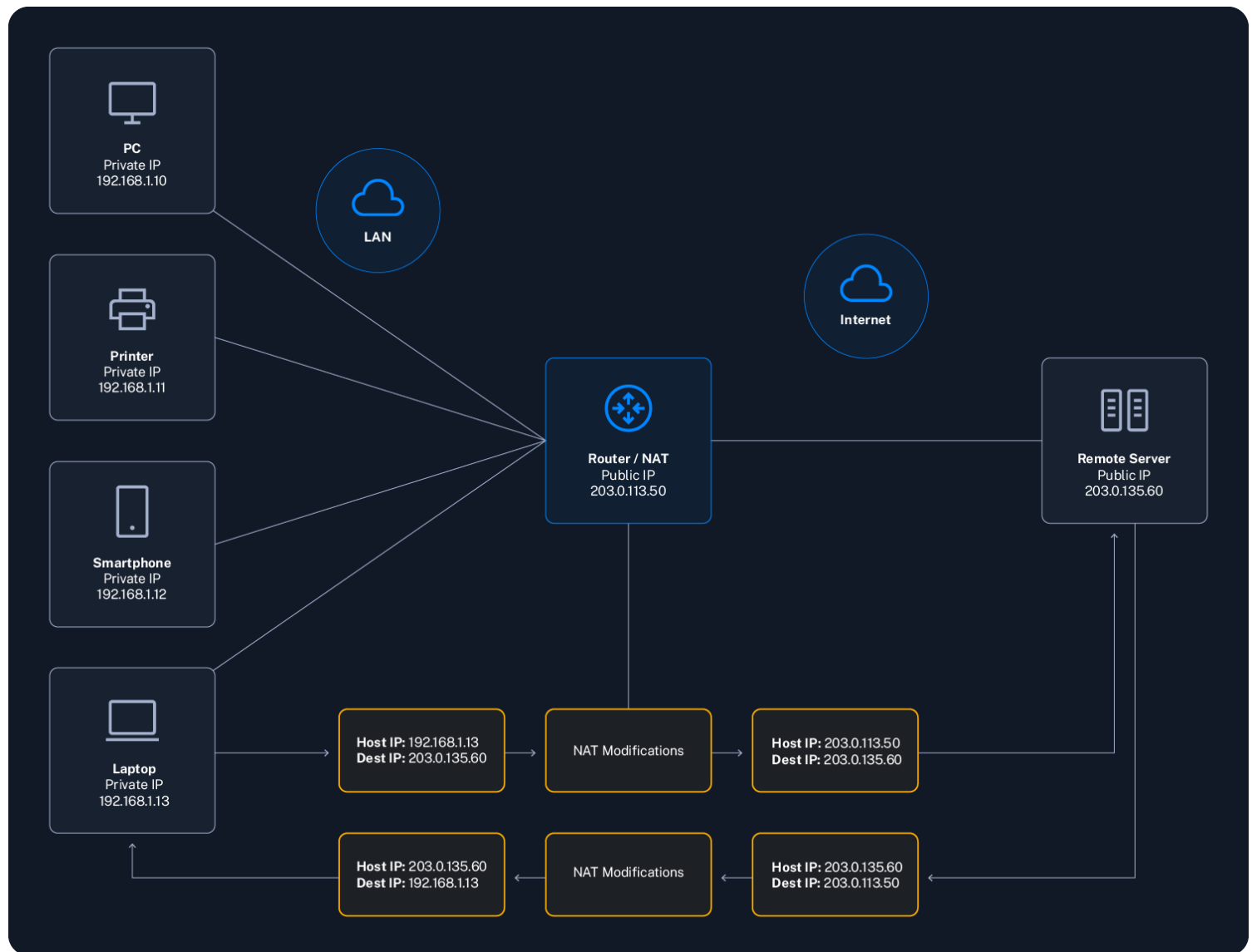
Network Address Translation

This shit is important.

So, turns out the **IPv4** pool is rather limited, nowadays (that's why **IPv6** is around). So, people found out that by reserving a set amount of **IPs** to be exclusively assigned internally as **IP** addresses (**RFC 1918**), assigning a public **IP** address to a router, and a way of mapping them to each other, we could preserve **IPv4** Addresses.

A device in a **LAN** gets attributed it's private **IP** by the router, and if anything on the internet feels like talking to that device, the packet must reach for the Router's public **IP**, and it's going to do a **NAT** function to translate a port into an internal IP + port.

Example:



Suppose the packet arriving back at the router is intended for 203.0.133.50:4444. The router, at some point, will have assigned incoming port 4444 packets to host 192.168.1.13's port 5555, for example. That's called **Port Address Translation (PAT)**

Here are some other **NATs**:

- **Static NAT**

- ◇ Involves a one-to-one mapping, where each private IP address corresponds directly to a public IP address.

- **Dynamic NAT**

- ◇ Assigns a public IP from a pool of available addresses to a private IP as needed, based on network demand.

This way a single Public IP address can correctly forward packets to it's local network's end devices.

I mean, that's all on NAT. If you want more info:

DNS

Domain Name System is, at the same time, complex but simple.

It is complex as a practical infrastructure component, sure. But it is quite simple to conceptually understand.

Imagine you had to remember the **IP** of every server that serves the web page you're looking for, instead of only it's name? Instead of "www.google.com", you'd have to remember one of their servers' **IP**, like for example 103.55.190.15 (completely made-up example).

DNS solves that by attributing names to **IPs**.

".com" is called a **Top-level Domain (TLD)**.

"google" is the **Domain** itself, also called a **2nd-Level Domain**.

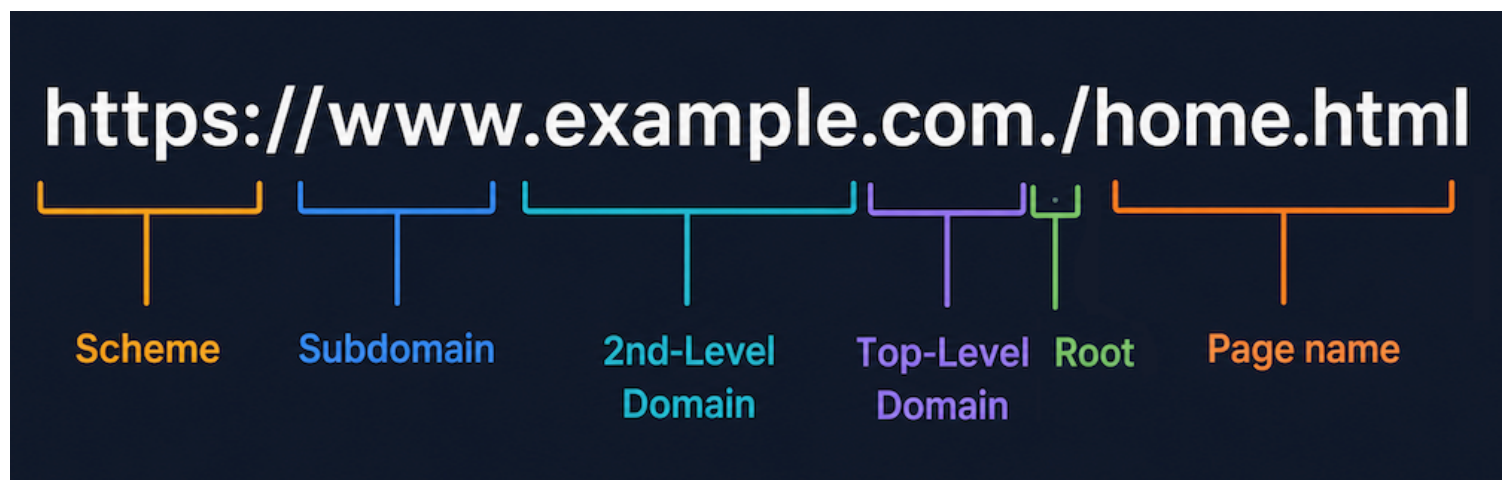
"www." is the **subdomain**.

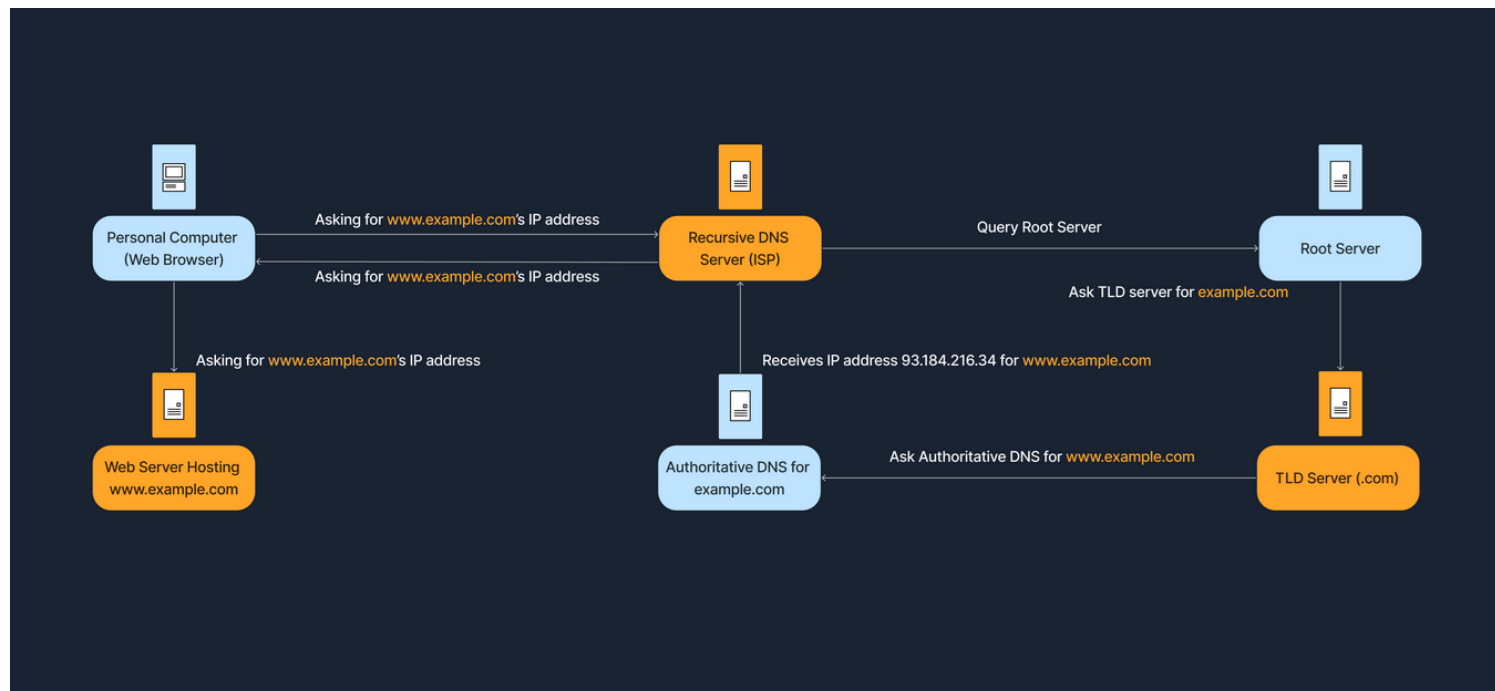
This information is stored locally on the End Device on it's DNS cache, but more realiably, on a DNS Server.

The **DNS** information is stored hierarchically.

So your computer first asks itself (**DNS Cache**) if it knows the **IP** for the **DNS** you're trying to send a request to. If doesn't itself have such knowledge, it asks the **ISP's Recursive DNS** server, which in turn asks the **Root Server**. This dude knows **TLD** Servers, so he asks them. The **TLD's DNS** server then asks the **Domain's Authoritative Name Server**, which finally responds with the **IP** of it's subdomain's server (where the sought after Application/service is hosted). The answers trickle back to each requester until finally reaching back to your computer.

Your computer may now send it's HTTP packet to the intended **IP**.



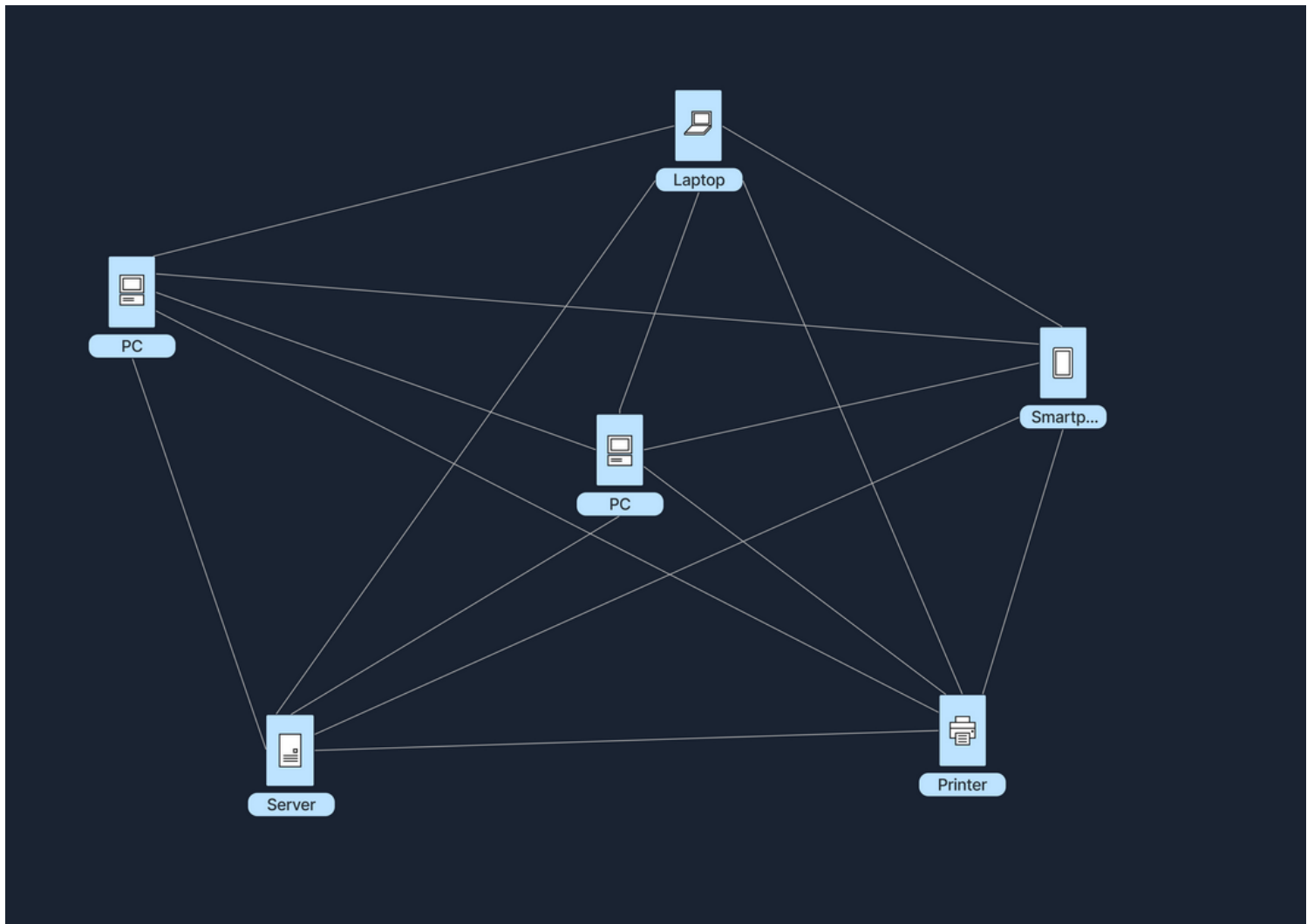


Internet Architecture

That's how communication schemes can be arranged with the use of the Internet as a means of delivering data.

Peer-to-peer

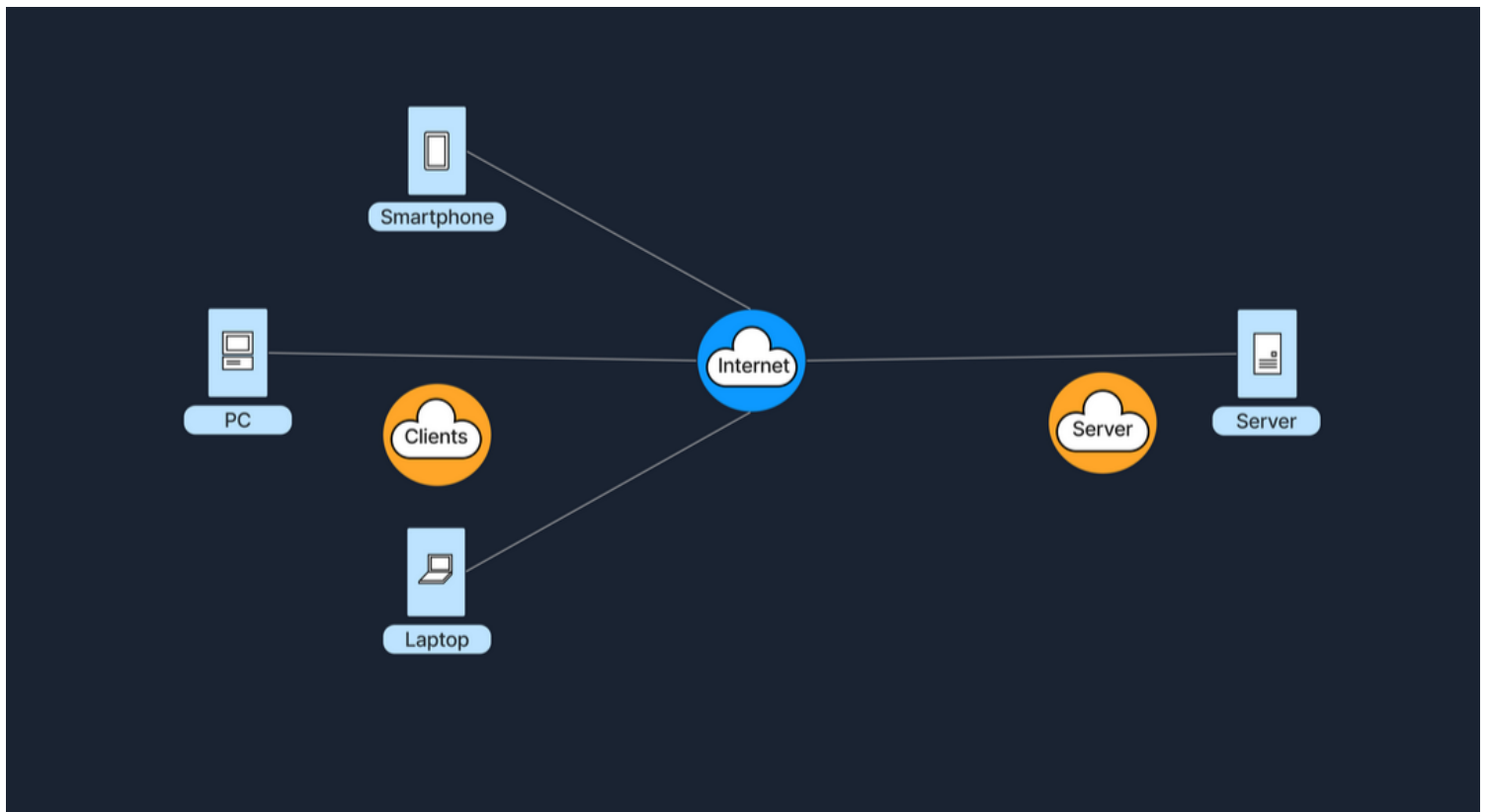
P2P is an architecture where every end device is connected to every other end device. So they send data directly to each other. This is great for efficiency and scalability, but woefully insecure.



Client-Server

A very popular model that centralizes data and processing in a server. Multiple hosts then request the same server for data or for a given application's functionality. This way we can exert more control over Data or Processing in general.

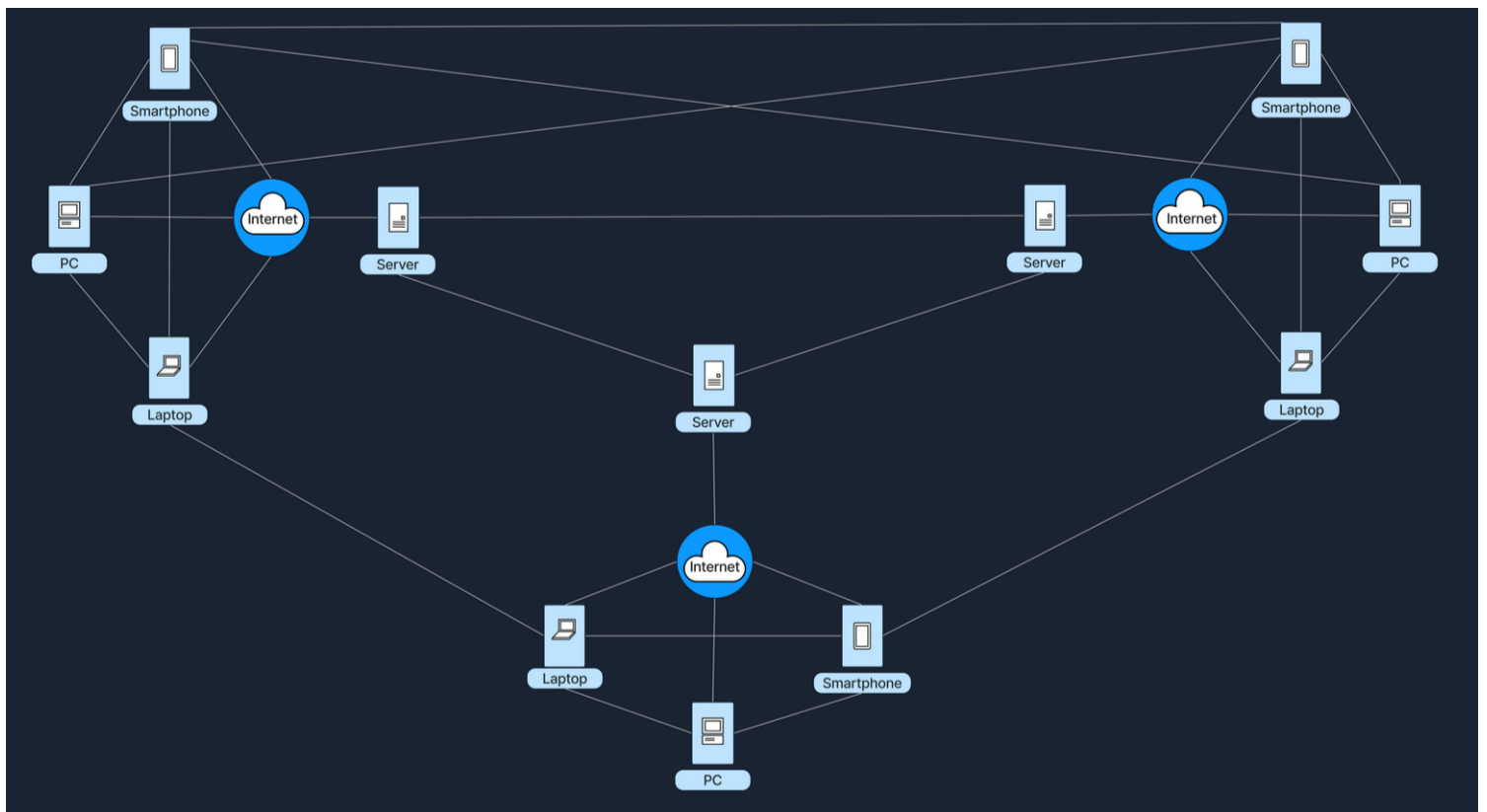
There are tiered approaches to this architecture, further separating the server's processing and data between different servers. For example, a two-tier architecture separates the presentation from the data, so a user has the view/interface of an App on his machine but the data is stored separately. The three-tier approach separates this further, placing a processing layer between the view and the data. This keeps going as much as you like with the N-tier architecture.



Hybrid Architecture

Is a mix of **P2P** and **Client-Server**.

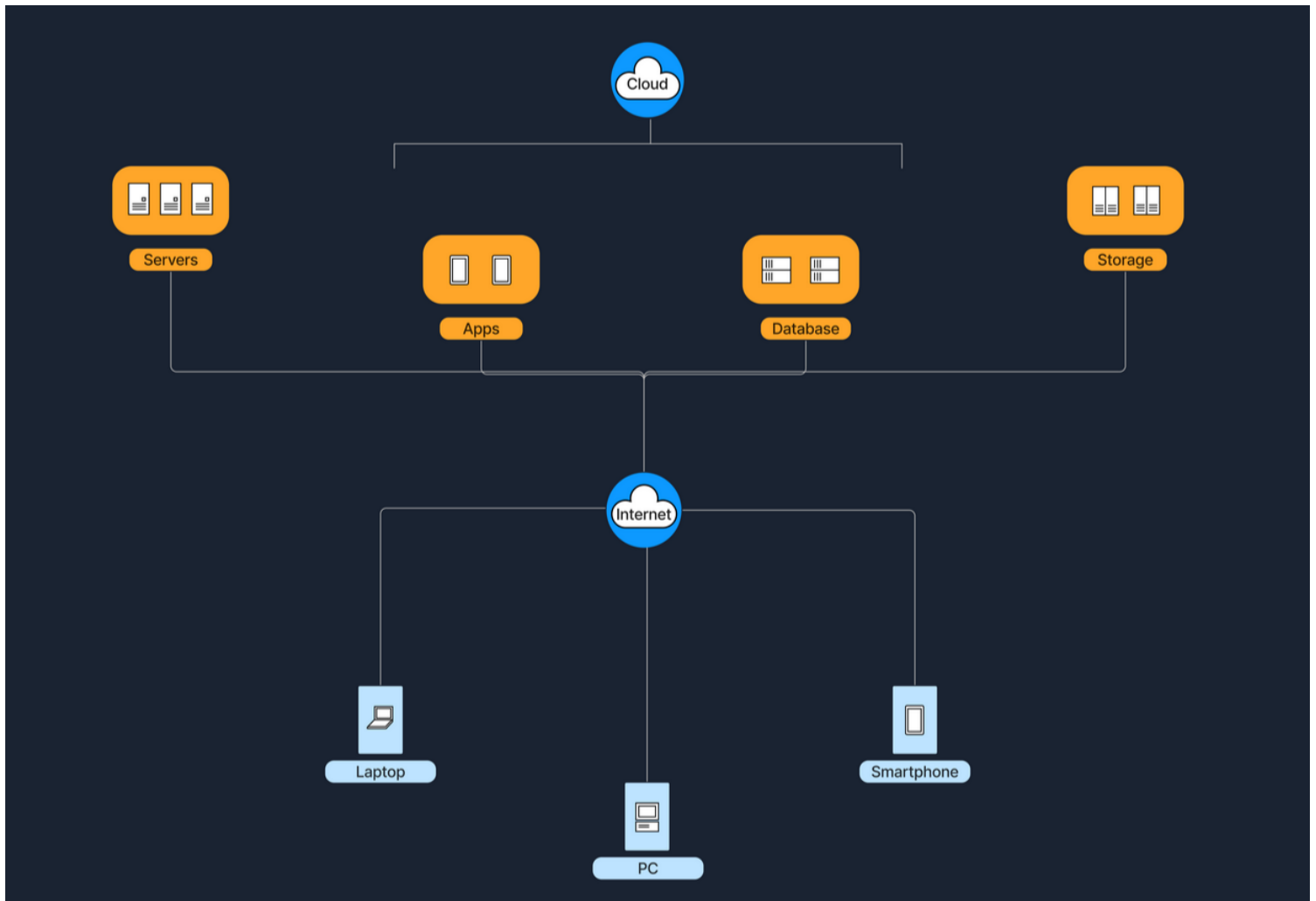
It is quite complex but can be very efficient and secure when implementing the best of both for it's desired application.



Cloud

Cloud is made to be ubiquitous, easy to setup, scalable, elastic, and well-measured. Which means that

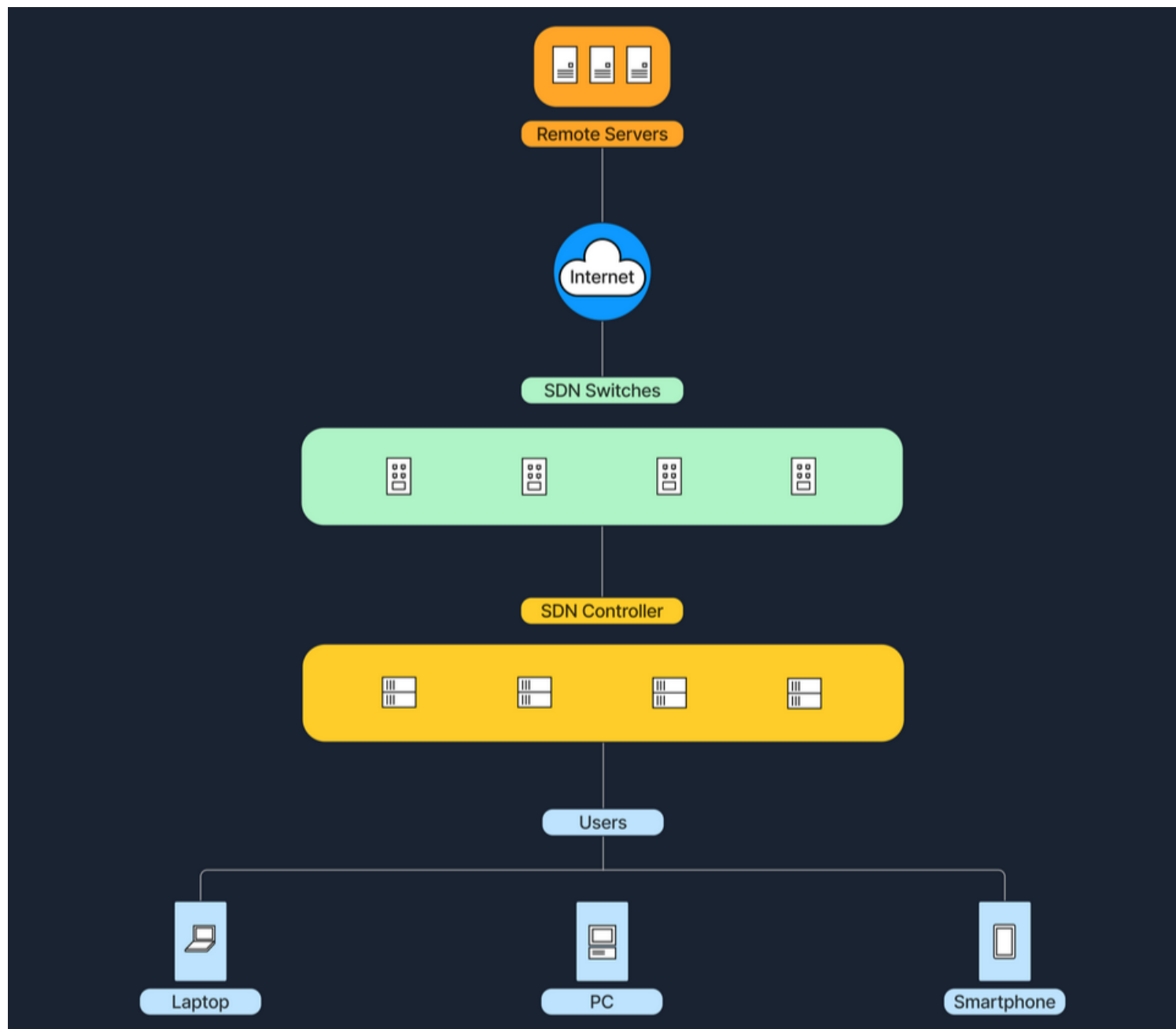
you can reach it through the Internet anywhere, you won't have much difficulty setting it up, it can scale as well as you need it to, even automatically, and the resources you use are tracked precisely, so you pay only what's been used. In a sense, it's akin to a well-ran Public Infrastructure Service such as water and/or electricity supplies.



Software-Defined Architecture

Software-Defined Networking (SDN) is a modern approach and it's confusing as f*ck to me. It separates the control plane, where decisions on traffic routing are made, from the data plane, where data actually flows in, which doesn't make any abstract sense to me at all.

Anyway, this is done in an Application level, in a way that the App even controls the Network layers, so as to dictate data flow to where it should go. It's easy to see this provides a programmable network management environment, so it's pretty flexible and a great network management simplifier, although it can get stupidly complex to implement and maintain.



In summary

Architecture	Centralized	Scalability	Ease of Management	Typical Use Cases
P2P	Decentralized (or partial)	High (as peers grow)	Complex (no central control)	File-sharing, blockchain
Client-Server	Centralized	Moderate	Easier (server-based)	Websites, email services
Hybrid	Partially central	Higher than C-S	More complex management	Messaging apps, video conferencing

Architecture	Centralized	Scalability	Ease of Management	Typical Use Cases
Cloud	Centralized in provider's infra	High	Easier (outsourced)	Cloud storage, SaaS, PaaS
SDN	Centralized control plane	High (policy-driven)	Moderate (needs specialized tools)	Datacenters, large enterprises

Wireless Networks

As you would've guessed, it's about Wi-Fi and mobile networks (Cellular).

Yeah, they're great for their mobility, but also for it's scalability and ease of installation.

Sadly, it's prone to interference and thus also security risks.

These wireless connections use **Radio Waves** and **Microwave** as the encoding of data, and air as the medium. These waves are emitted at specific frequencies measured in **Hertz (Hz)**.

Common Frequency Bands

- 2.4 GHz (Gigahertz)
 - ◊ Used by older Wi-Fi standards (802.11b/g/n).
 - ◊ Better at penetrating walls, but can be more prone to interference (e.g., microwaves, Bluetooth).
- 5 GHz
 - ◊ Used by newer Wi-Fi standards (802.11a/n/ac/ax).
 - ◊ Faster speeds, but shorter range.
- Cellular Bands
 - ◊ For 4G (LTE) and 5G.
 - ◊ These range from lower frequencies (700 MHz) to mid-range (2.6 GHz) and even higher frequencies for some 5G services (up to 28 GHz and beyond).

Due to the nature of Wireless Frequencies and their importance, governments and other institutions often regulate Radio Wave frequencies in order to ensure orderly use of the airwaves.

Network Security

Turns out it's necessary.

In order to help maintain the **Infosec Triad** intact, **Firewalls** and **Intrusion Detection/Prevention Systems (IDS/IPS)** are required.

Firewall

A device that filters traffic based on an **Access Control List (ACL)**, which lists what **Protocols** can pass through, and also to which **IPs** and **Ports** they can go to, depending on where they come from (**IP**).

It either allows or block traffic.

Types of Firewall

There's a few of them, but the concept is simple: Filter traffic based on a set of criteria.

- **Packet Filtering Firewall**

- ◇ Operates at Layer 3 (Network) and Layer 4 (Transport) of the OSI model.
- ◇ Examines source/destination IP, source/destination port, and protocol type.
- ◇ Example: A simple router ACL that only allows HTTP (port 80) and HTTPS (port 443) while blocking other ports.

- **Stateful Inspection Firewall**

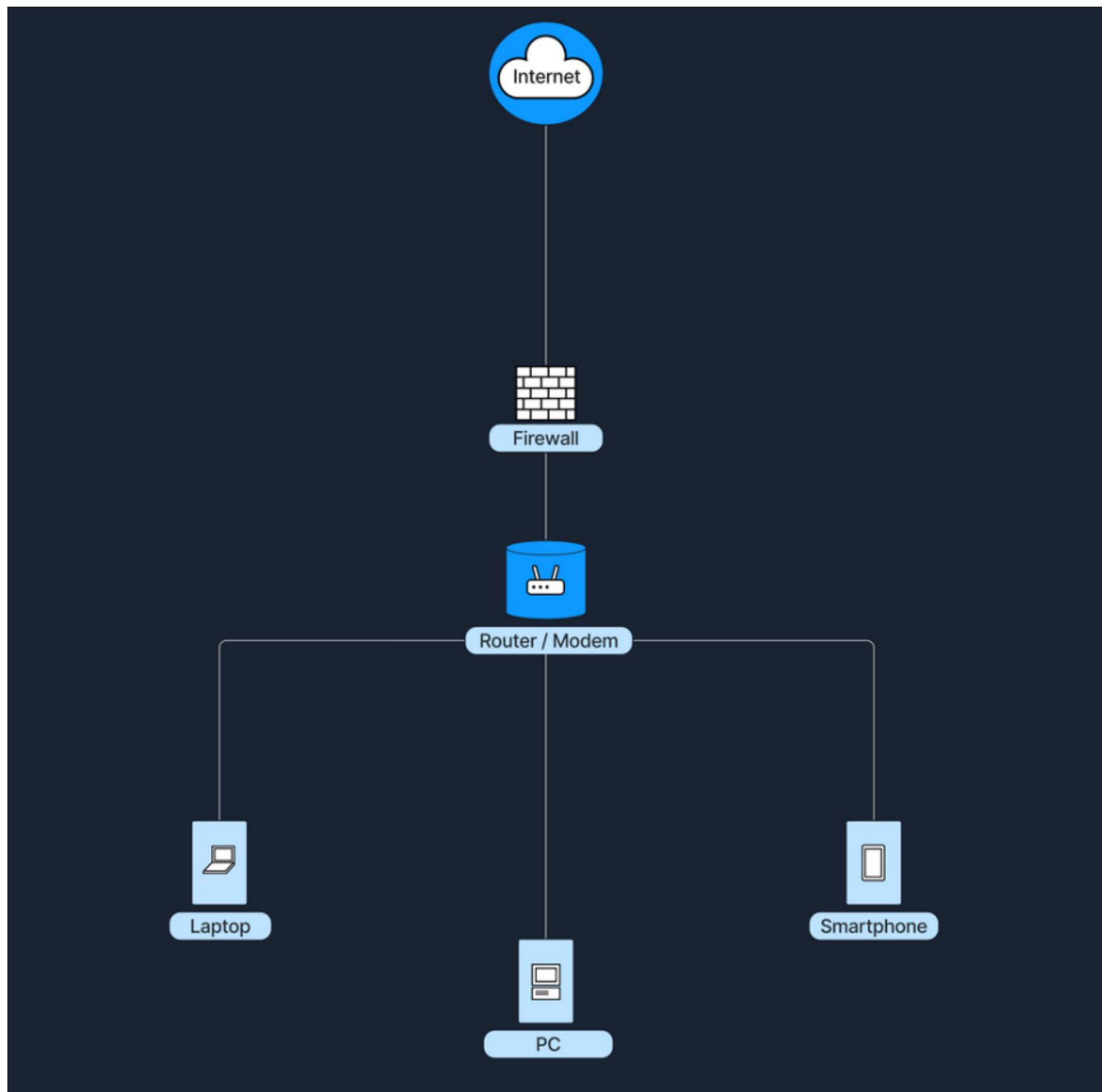
- ◇ Tracks the state of network connections (Sessions).
- ◇ More intelligent than packet filters because they understand the entire conversation.
- ◇ Example: Only allows inbound data that matches an already established outbound request.

- **Application Layer Firewall (Proxy Firewall)**

- ◇ Operates up to Layer 7 (Application) of the OSI model.
- ◇ Can inspect the actual content of traffic (e.g., HTTP requests) and block malicious requests.
- ◇ Example: A web proxy that filters out malicious HTTP requests containing suspicious patterns.

- **Next-Generation Firewall (NGFW)**

- ◇ Combines stateful inspection with advanced features like deep packet inspection, intrusion detection/prevention, and application control.
- ◇ Example: A modern firewall that can block known malicious IP addresses, inspect encrypted traffic for threats, and enforce application-specific policies.



Intrusion Detection/Prevention Systems (IDS/IPS)

Security solutions designed to monitor and respond to suspicious activity in a network or device.

IPS's usually work proactively, blocking stuff. IDS's are shier and usually only log and alert when stuff is happening.

Both analyze traffic to figure out if anything suspicious is going on, usually by comparing them to a database of know malicious traffic patterns and signatures. In fact, there are two approaches:

Signature-based detection -> Matches traffic against a database of known exploits.

Anomaly-based detection -> Detects anything unusual compared to normal activity.

And two different distributions:

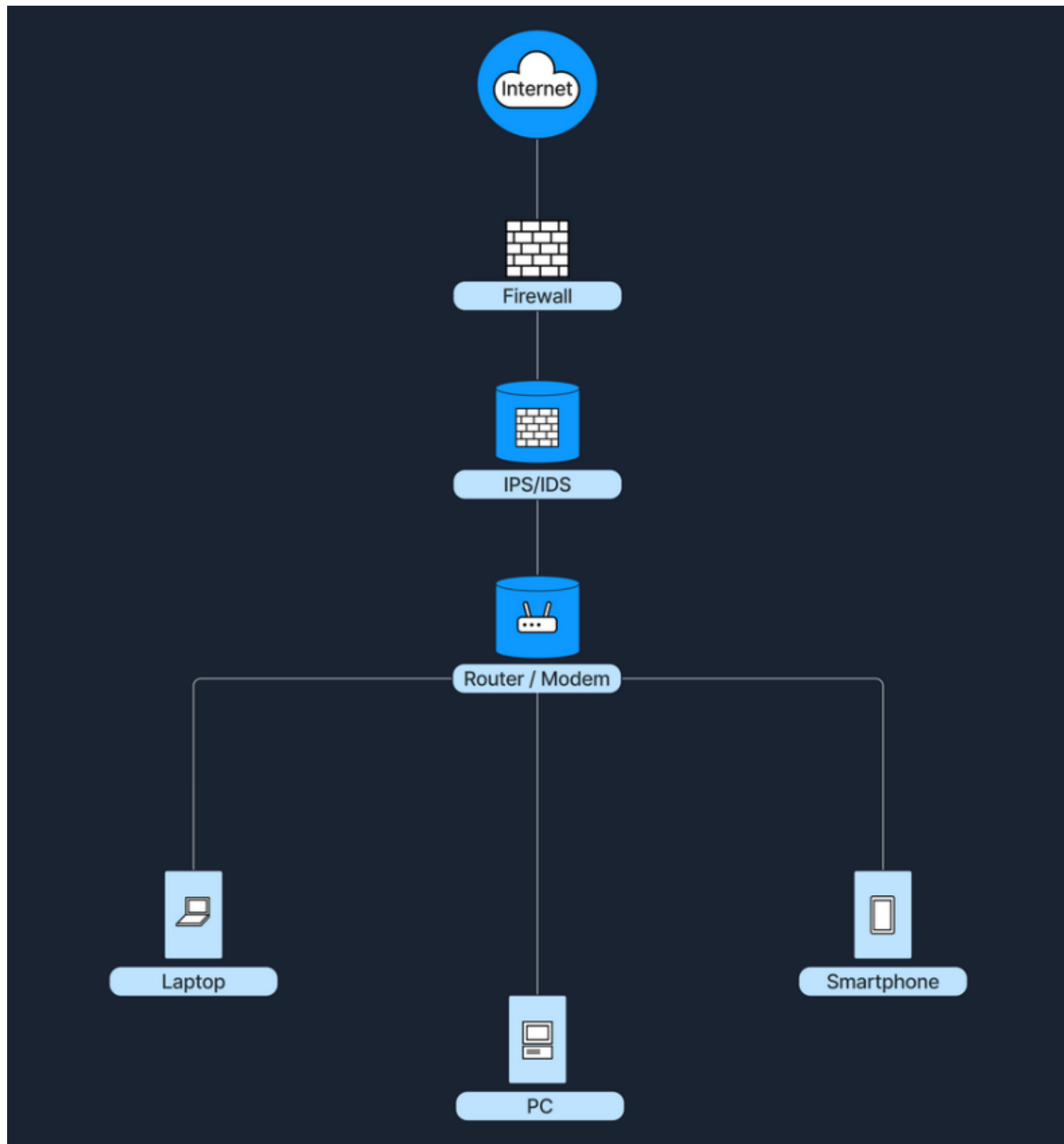
- Network-Based IDS/IPS (NIDS/NIPS)

- ◊ Hardware device or software solution placed at strategic points in the network to inspect all

passing traffic.

- ◇ Example: A sensor connected to the core switch that monitors traffic within a data center.
- Host-Based IDS/IPS (HIDS/HIPS)
 - ◇ Runs on individual hosts or devices, monitoring inbound/outbound traffic and system logs for suspicious behavior on that specific machine.
 - ◇ Example: An antivirus or endpoint security agent installed on a server.

There are some strategic ways to position **IDS/IPS**'s to be the most effective. They can be placed right after where all the traffic arrives and leaves from (such as behind the firewall), so it scans everything that goes in and out. They can also run in hosts.



Best Practices for Network Security

- Define Clear Policies Consistent firewall rules based on the principle of least privilege (only allow what is necessary).
- Regular Updates
 - ◇ Keep firewall, IDS/IPS signatures, and operating systems up to date to defend against the latest threats.
- Monitor and Log Events
 - ◇ Regularly review firewall logs, IDS/IPS alerts, and system logs to identify suspicious patterns early.
- Layered Security
 - ◇ Use defense in depth (a strategy that leverages multiple security measures to slow down an attack)

with multiple layers: Firewalls, IDS/IPS, antivirus, and endpoint protection to cover different attack vectors.

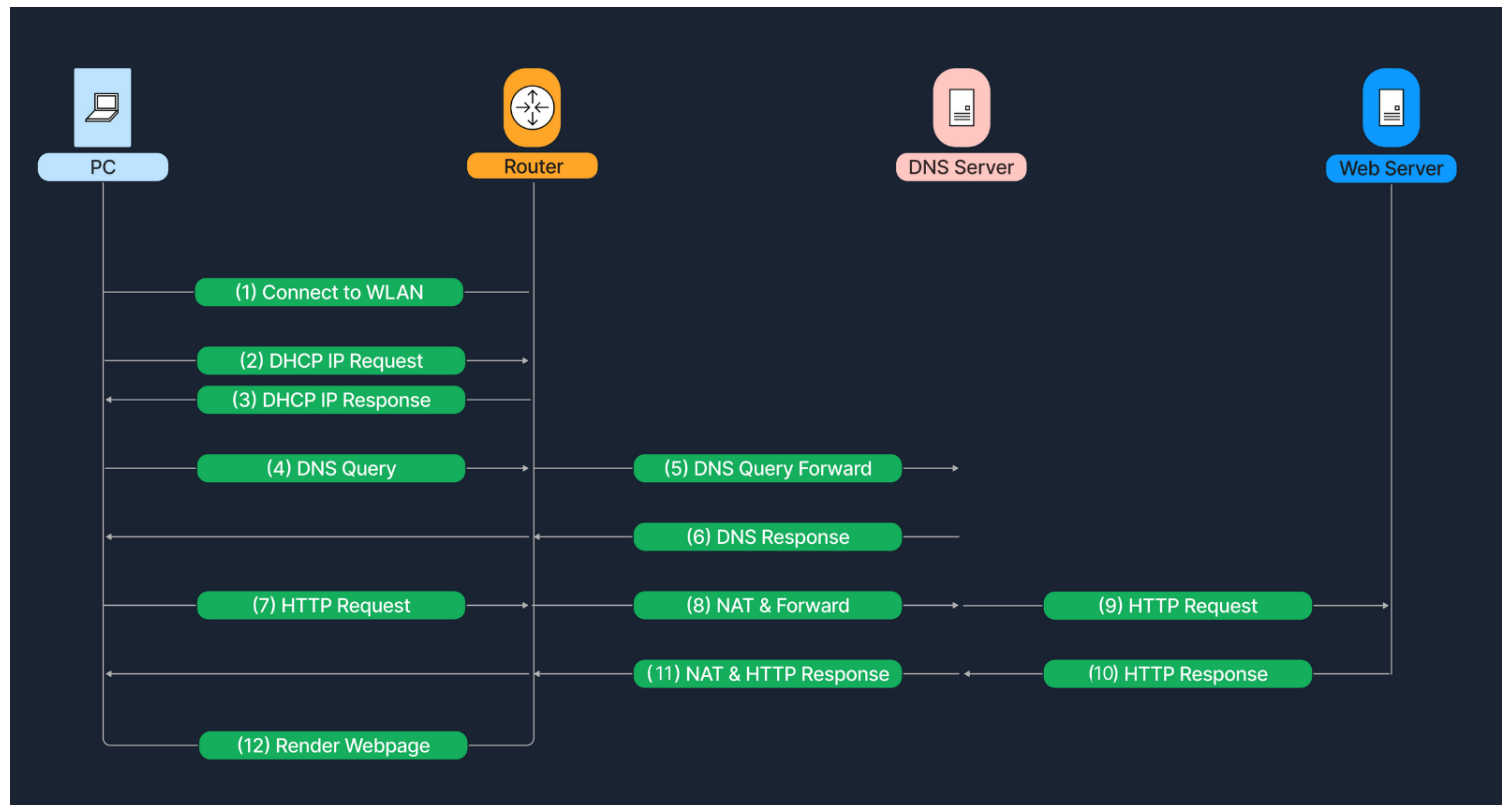
- Periodic Penetration Testing

- ◇ Test the effectiveness of the security policies and devices by simulating real attacks.

Data Flow Example

[Data Flow Example | Hack The Box Academy](#)

But the following image is a good summary, even without much text:



Interactive Skills Assessment

After all that, comes the time for some actual practice.

Chapter 1

The `ifconfig -a` command was responsible for showing me the network interfaces available. The flag `-a` made sure I could see all of them.

There, some of them were of greater interest:

- lo

- ◇ The **loopback interface**. That's the interface used for hosts that send data to themselves.
- ◇ The default **IP** is 127.0.0.1, also known as localhost.

- ens3
 - ◇ The actual **public interface** for the lab's pwnbox. It has a public **IP** address to which we connected.
- tun0
 - ◇ This interface represents a **private tunnel** made by a **VPN**.
 - ◇ All HTB machines spawn in the same private network in order to be interreachable, the interface in charge of establishing that communication is tun0.

netstat -tulnp4 showed me both the active and inactive TCP/UDP connections in an IP:PORT configuration.

netstat -tulp4 translated the **IP's** and **Ports** to form a hostname:service pair.

With those, I could now see some services running. Particularly interesting were the **ports 80** and **631**. These are **default ports** for **HTTP** and **VNC**, respectively, most known for allowing browser-based visualization of data (**HTTP**) and remote screen sharing and remote access (**VNC**). Which explains how we can access the machine remotely, via our browser.

My jaw kinda dropped at this, really. I'm not sure why, but I got quite flabbergasted at this realization.

Also, **ens3** translated to the machine's hostname, which showed me a different use for **DNS, Reverse DNS**.

Total jaw drop.

Chapter 2

Spawning another system, came the time to test out actual networking functionalities.

ip route get <target ip> Showed me the route taken from the Pwnbox to the target machine, which was through the tun0 interface.

ping -c 4 <target ip> Showed me I **could** reach the target from the Pwnbox. Although I'm used to pings at this point, I ended up realizing that being able to see it was one thing, but being able to reach it was another. Should have been obvious, but at least now I know it is a possibility.

nmap <target IP> began the enumeration of open TCP ports on the target, which showed me that **FTP (Port 21)** and **HTTP (Port 80)** were open. Others were as well, but only those 2 were important to this lesson.

An interesting observation is the ability to conclude the machine you're scanning is a **Windows**-based host from the 135, 139 and 445 **ports**. I had no idea but apparently it is common for Windows-based devices to have these ports **open by default**. 3389 and 5357 were also open but were less relevant than the other 3 for identifying a Windows.

Chapter 3

Being able to reach the other host and having an idea of what's running and listening for connections, came the "poking around and getting in" part.

I'd never done anything like this by myself, but surprisingly, it was fairly easy. Particularly because my hand was being held like I was a child. But even so, it could have been complicated enough for me to not understand a thing or feel absolutely lost, even if being guided through it. That said, it was not the

case, and I thoroughly enjoyed it!

`nmap -p21,80 -sC -sV <target ip>` focused the **nmap** on the **HTTP** and **FTP ports**. With it we got info about the **FTP** service's authentication config, which allowed anonymous access. The **HTTP** service, on the other hand, was filtering out our request.

`nc <target ip> 21` created a raw **TCP** connection with the **FTP port** on the Target. Loggin in as anonymous we initiated **passive mode**, allowing data to be served from the **FTP** server in a dynamically set port, for which you actually have to solve a "big enough to be a hurdle" math operation to be able to get.

`nc -v <target ip> <dynamic port>` then set us up for success, connecting the Pwnbox to the target's open **FTP data channel**. To be fair, I don't quite understand why we weren't able to use **RETR** at this point. Instead we had to:

- `LIST[Ctrl+V][Enter][Enter]` and `PASV[Ctrl + V][Enter][Enter]`
 - ◇ On the **Connection channel**
- `nc -v <target ip> <dynamic port>` again, opening a **data channel**.
 - ◇ From a different terminal.
- Followed by `RETR Note-From-IT.txt[Ctrl+V][Enter][Enter]`
 - ◇ On our new **data channel**.

And with that we actually got the "Note-From-IT.txt" file printed out on the terminal. Reading it we learn about the specific **User-Agent** required as an **HTTP header** on any request to the **HTTP** service running on the target. That is the key to not get our requests filtered anymore.

`nc -v <target ip> 80` opened a TCP session with the **HTTP** service, and by building the following HTTP Request:

```
GET / HTTP/1.1
Host: <target ip>
User-Agent: Server Administrator
```

I got the server response. With an IIS interface in HTML. And with another something at the end. A flag. My first flag.
HTB{S00n_2_B_N3tw0rk1ng_GURU!}

A historical moment for me.

Conclusion

This was a great experience. The module was advertised as 4 hours long, but it took me from 3p.m. to 11:30p.m. to finish it. Of course I took some breaks, even to cook and eat dinner. I also had some moments where I wasn't really being able to pay attention to the module, and was taking these notes from the get-go.

SO yeah, you could possibly do it faster if more well distributed and not being so thorough with your

notes (especially visible at the first chapters of this document).

Anyways, great module. It was nice to review all of these fundamentals. I've also definitely learned many new things and got back a bit of the ability to visualize Computer Networks conceptually.

I think I'm too tired to go over any more details, I've definitely typed out a lot of them in here already so, count their existence as a part of this conclusion.

Until next time.

Dominus Tecum, Virgo Serena.